

UNIVERSITÀ  
DEGLI STUDI  
DI PADOVA

# N-Body Simulation

Design, Optimizations and Benchmarking

M. Sc. Physics of Data, University of Padova

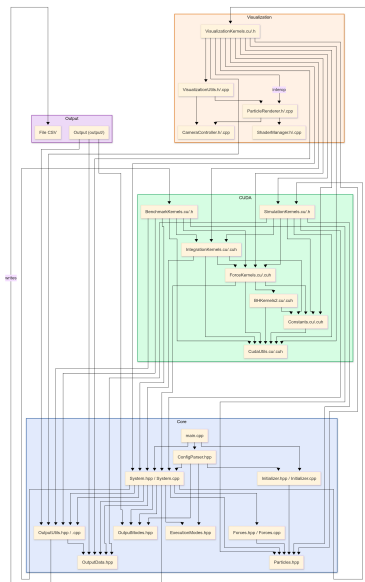
Alessio

September 9, 2025

1. Introduction and Motivation
2. Project Evolution
3. Memory Management and Optimizations
4. Force and Integration Methods
5. Performance Comparison with Existing Codes
6. Limitations and Lessons Learned
7. Conclusions

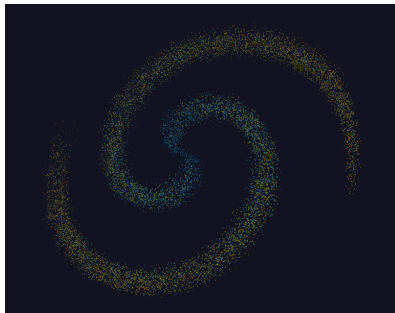
- Started as a single-file C++ code for Voyager II trajectory
- Evolved into a modular, configurable N-body simulator
- Main goals:
  - Flexible and extensible framework
  - Fair CPU vs GPU performance comparison
  - Advanced optimizations and interactive visualization
  - Something ready for researchers to use
  - Performance on par or better than existing pro solutions

- Initial single-file C++ implementation
- Output separation and modularization
- Integration of OpenMP, CUDA, (AVX2)
- CMake build system (Container building for portability), JSON configuration
- Interactive visualization with OpenGL

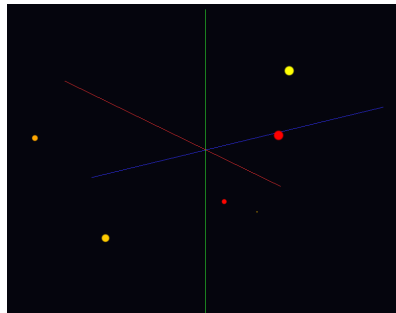
UNIVERSITÀ  
DEGLI STUDI  
DI PADOVA

- **Core:** simulation logic, data structures, and CPU part
- **CUDA:** GPU kernels, utilities and host functions
- **Visualization:** OpenGL and OpenGL/CUDA interop
- **Output:** CSV output, dynamical buffer to use for other integrations
- **Configuration:** single JSON file

- Single JSON configuration file
- Flexible initialization: random, file-based, or custom implementations (galaxy, stellar systems, etc.)



Example of a galaxy configuration



Example of random initialization

- Struct of Array of Small Structures (SoAoSS) for efficient access and fully coalesced memory with double4
- Pinned memory and CUDA streams
- Shared memory tiling in GPU kernels:  $\sim 768/1024^+$  particles per block,  $> 2$  blocks per SM (with auto-blocksize)
- AVX2 vectorization and OpenMP threading
- Low-ish memory footprint: 1.9 GiB for 10M particles
- In CSV output mode, System is copied only at the start  $\rightarrow$  multiple cuda streams for memory and compute management.
- In visualization mode, System is copied every frame to a dynamic buffer, which is then used for rendering  $\rightarrow$  worst case scenario  $\Rightarrow$  OpenGL-cuda interop and separate rendering stream  $\rightarrow$  less bottleneck. (WIP)

- Pairwise, Adaptive Mutual Softening, Barnes-Hut (no GPU implementation)
- Configurable physical constants and softening
- Multiple integration schemes: Euler (not symplectic), Velocity Verlet (symplectic)

**Pairwise Newtonian:**

$$\vec{a}_i = G \sum_{j \neq i} m_j \frac{\vec{r}_j - \vec{r}_i}{|\vec{r}_j - \vec{r}_i|^3}$$

**Adaptive Mutual Softening:**

$$\vec{a}_i = G \sum_{j \neq i} m_j \frac{\vec{r}_j - \vec{r}_i}{(|\vec{r}_j - \vec{r}_i|^2 + \epsilon_{ij}^2)^{3/2}}$$
$$\epsilon_{ij} = \max \left( \eta |\vec{r}_j - \vec{r}_i| \left( \frac{m_i + m_j}{3\langle m \rangle} \right)^{1/3}, \epsilon_{\min} \right)$$

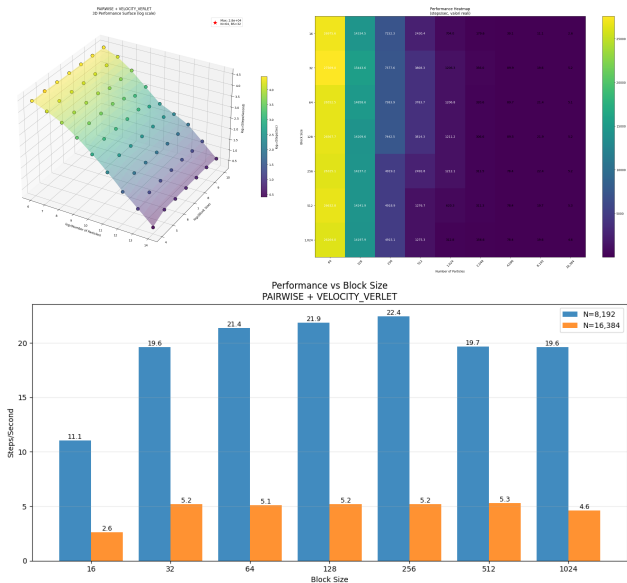
where  $G$  is the gravitational constant,  $\epsilon_{ij}$  is the adaptive softening,  $\eta$  and  $\epsilon_{\min}$  are tunable parameters, and  $\langle m \rangle$  is the average mass.



# Block-Size Comparison Benchmark



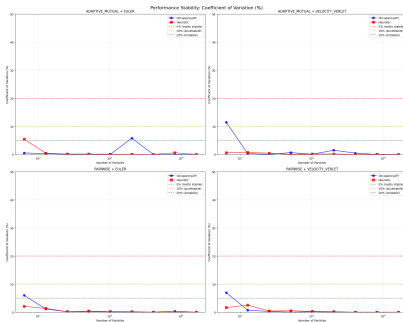
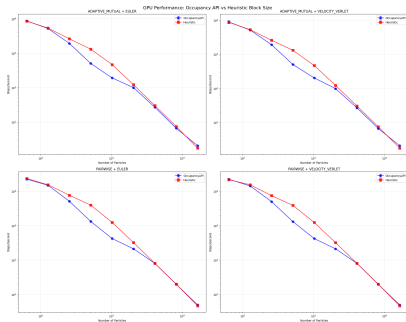
UNIVERSITÀ  
DEGLI STUDI  
DI PADOVA



# Block Size Auto – Tuning



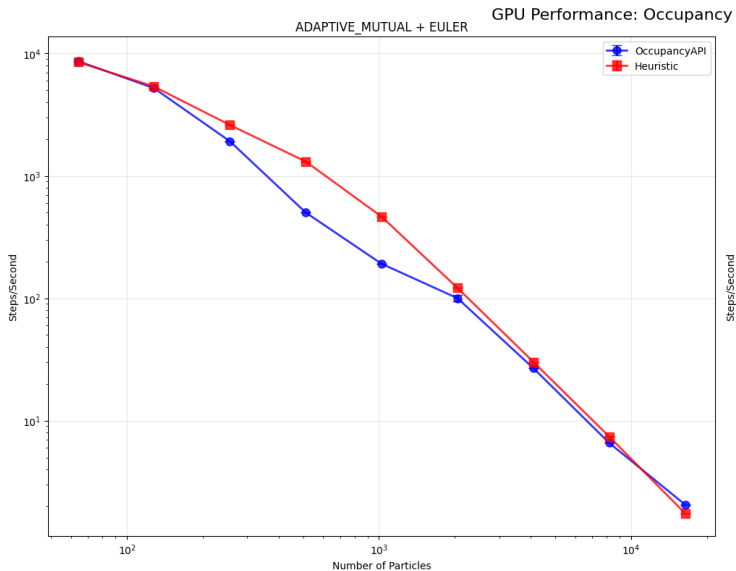
- **Block size selection** is crucial for GPU efficiency in these types of problems.
- Two strategies compared:
  - **Occupancy API** (NVIDIA)
  - **Heuristic** (custom, optimized for this workload)



# Block Size Auto – Tuning



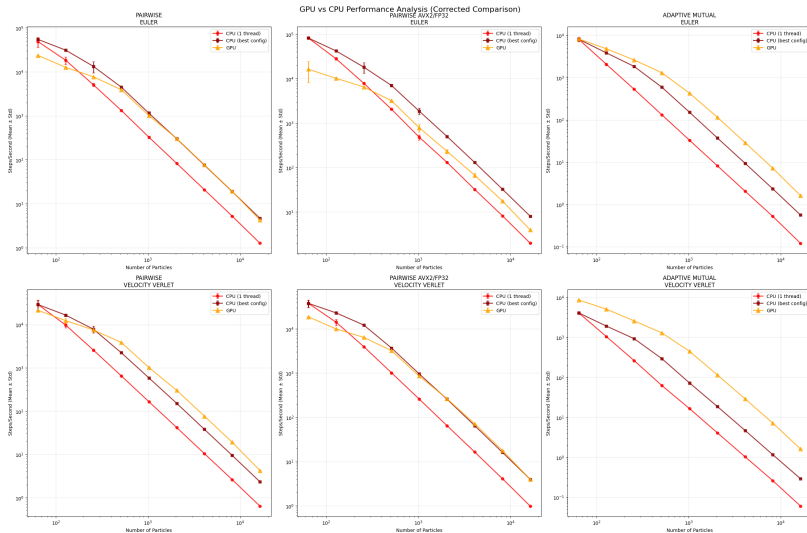
UNIVERSITÀ  
DEGLI STUDI  
DI PADOVA



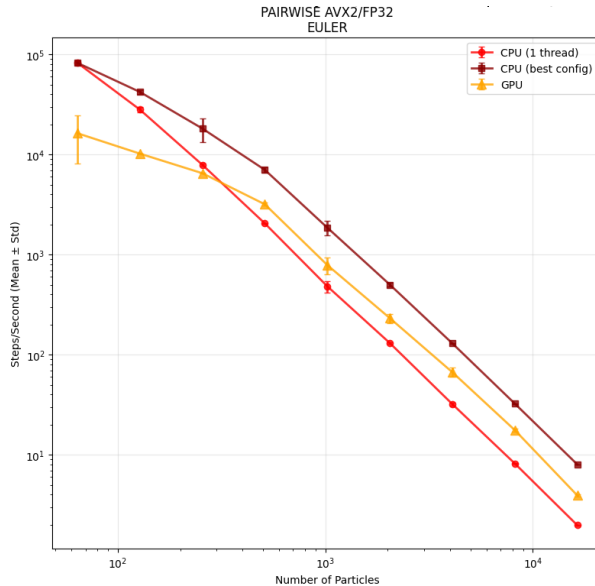


12 of 26

# Basic CPU – GPU Comparison Benchmark



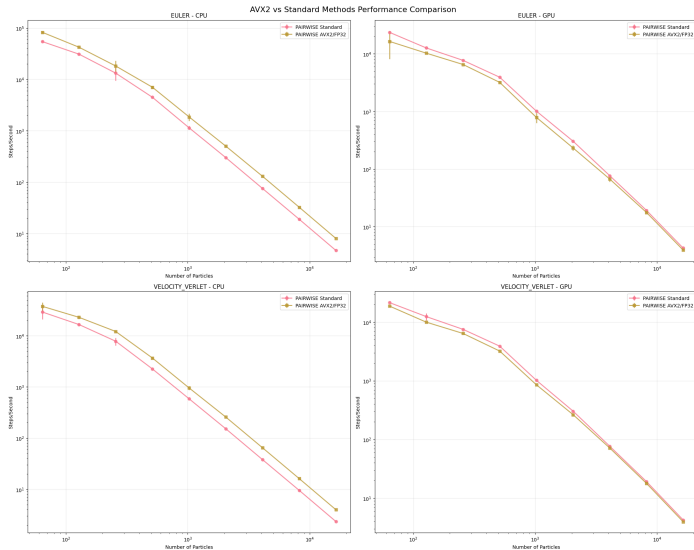
# Basic CPU – GPU Comparison Benchmark



# Basic CPU – GPU Comparison Benchmark



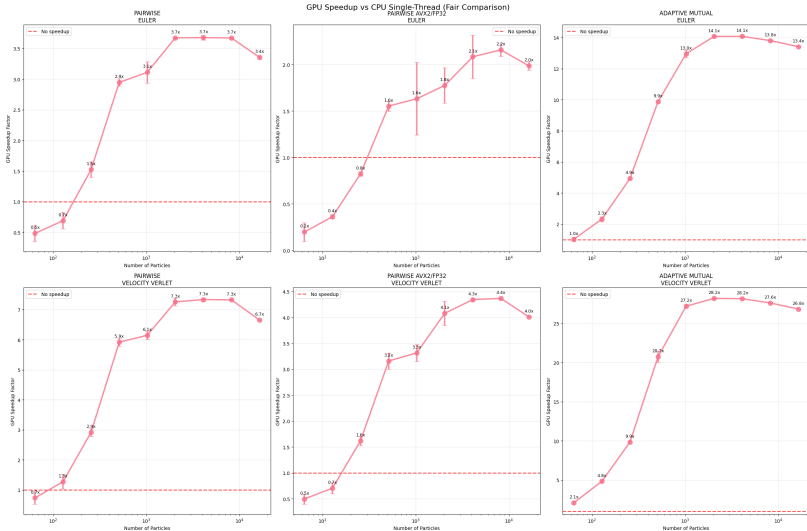
UNIVERSITÀ  
DEGLI STUDI  
DI PADOVA



# Basic CPU – GPU Comparison Benchmark



UNIVERSITÀ  
DEGLI STUDI  
DI PADOVA

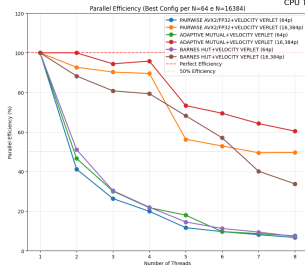




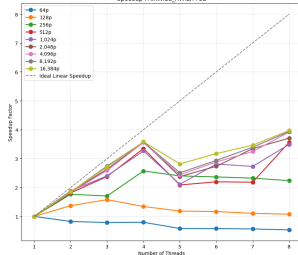
# CPU – Methods Comparison Benchmark



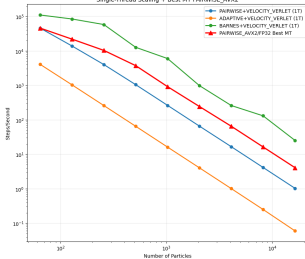
CPU Threading Efficiency Analysis



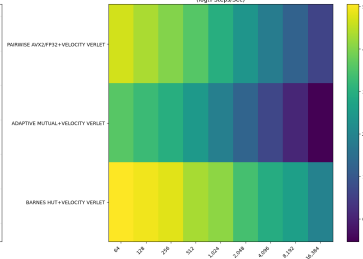
Speedup PAIRWISE\_AWX2/FP32



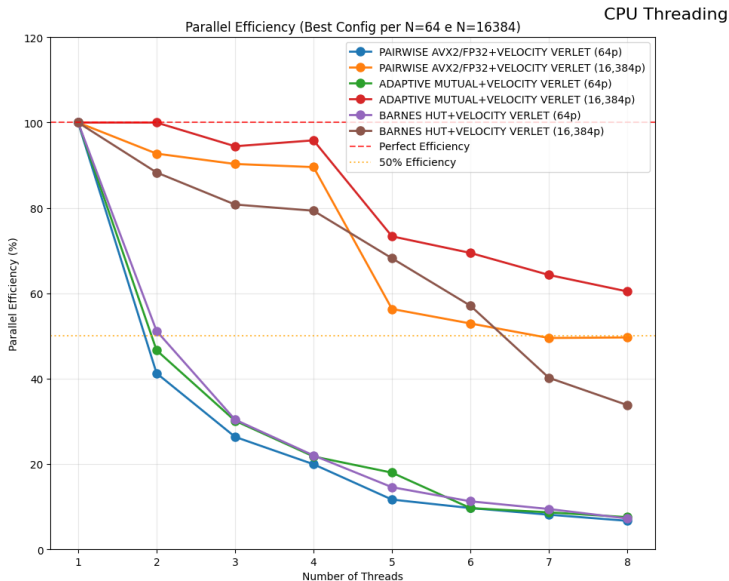
Single-Thread Scaling + Best MT PAIRWISE\_AWX2



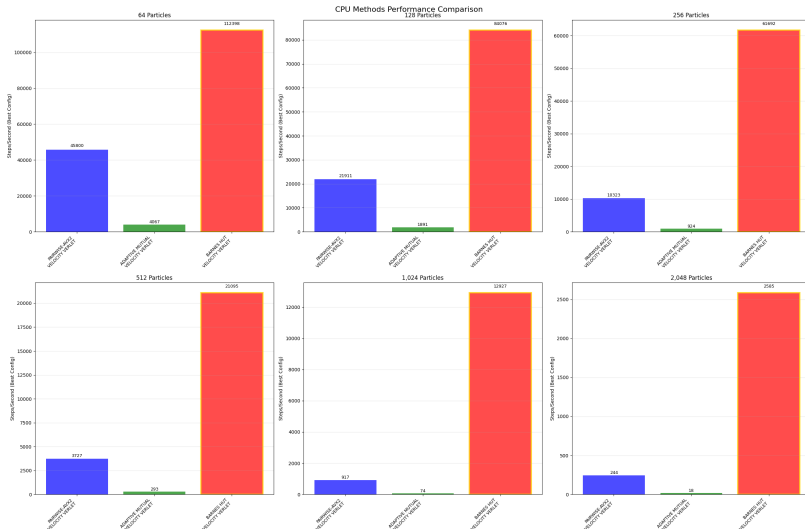
Best Performance Heatmap  
(log<sub>10</sub> Steps/Sec)



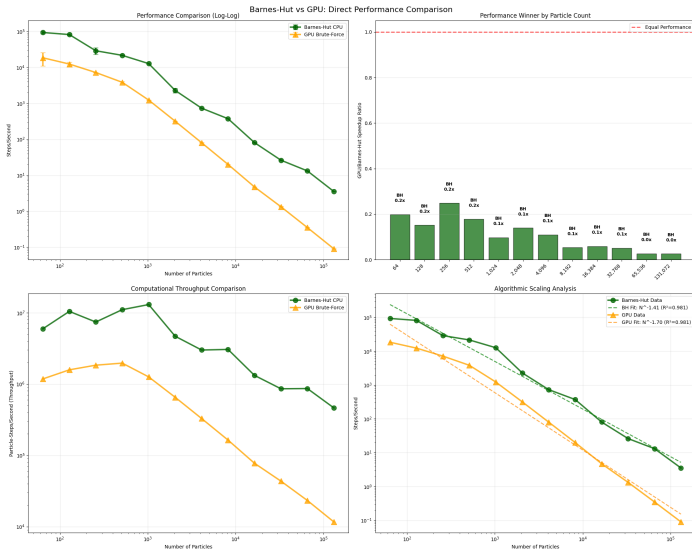
# CPU – Methods Comparison Benchmark



# CPU – Methods Comparison Benchmark



# BH vs GPU Comparison Benchmark



# Pairwise Performance Comparison



| Code            | Hardware     | $N = 10^3$      | $N = 10^4$         | $N = 10^5$           |
|-----------------|--------------|-----------------|--------------------|----------------------|
| This work (GPU) | GTX 1050 Ti  | $1 \times 10^3$ | $2 \times 10^1$    | $1.8 \times 10^{-1}$ |
| This work (CPU) | i7-7700HQ    | $1 \times 10^3$ | $1 \times 10^1$    | $1.1 \times 10^{-1}$ |
| REBOUND (CPU)   | Xeon E5-2680 | $1 \times 10^1$ | $5 \times 10^{-1}$ | $1 \times 10^{-2}$   |
| NBODY6++ (GPU)  | Tesla K40    | $1 \times 10^3$ | $1 \times 10^2$    | 1                    |
| PeTar (GPU)     | V100/A100    | $1 \times 10^4$ | $1 \times 10^3$    | $1 \times 10^1$      |

FPS  $\approx$  steps per second.

Orders of magnitude, for qualitative comparison only.

# Barnes – Hut (CPU) Performance Comparison



UNIVERSITÀ  
DEGLI STUDI  
DI PADOVA

| Code            | Hardware     | $N = 10^3$      | $N = 10^4$      | $N = 10^5$      |
|-----------------|--------------|-----------------|-----------------|-----------------|
| This work (CPU) | i7-7700HQ    | $1 \times 10^4$ | $5 \times 10^2$ | $1 \times 10^1$ |
| REBOUND (CPU)   | Xeon E5-2680 | $1 \times 10^2$ | $1 \times 10^1$ | 1               |
| GADGET-2 (CPU)  | Xeon E5-2680 | $1 \times 10^2$ | $1 \times 10^1$ | 1               |

FPS  $\approx$  steps per second.

Orders of magnitude, for qualitative comparison only.

- Bottlenecks in compute for very large  $N$  (different methods, emulation, ...)
- Tradeoffs between accuracy and speed (double in GPU, same memory pattern in CPU)
- Importance of a clear vision at the start of the project
- Need for a robust testing framework
- Make only one main entry point per external library (or as few as possible)

- A running program you can compile without looking at any library requirements (nearly done with Docker+CMake)
- Make Barnes-Hut work on GPU (not trivial)
- Implement more advanced methods (e.g. FMM, TreePM, etc.)
- Explore ML/DL approaches:
  - Surrogate models for fast prediction of N-body dynamics
  - ML-based force approximation or parameter tuning
  - Graph Neural Networks for particle interactions
  - Anomaly detection in simulation outputs
- Try "naive" implementation on new specialized hardware/architectures (AVX-512, TPU, Tensor cores etc.)
- Mixed precision support on CPU/GPU
- Voxel-based/Ray traced/compute shader rendering for large N
- Better initialization methods (e.g. Plummer, King models for star clusters)
- Integration with astrophysical libraries (e.g. AMUSE, Astropy)



- We skipped many small details (tiling, async memory...) in this presentation, so if you want to know more, please check the GitHub repository or ask me :)
- High-performance, modular N-body simulation achieved
- GPU acceleration enables large-scale experiments of:
  - $\sim 10^{3\sim 4}$  particles in real time ( $> 30$  steps/s or FPS)
  - $\sim 10^{5\sim 6}$  particles not in real time
- Framework ready for extension and research

Thank you for your attention!

Questions?

Demo?